

OPEN ENDED LAB (OEL)

EL-421 (Embedded Electronics)

GROUP MEMBERS:

IBRAHIM KHAN (EL-20019)

FOUZAN BAIG (EL-20029)

MUHAMMAD ABDUL AHAD (EL-20136)

SECTION: A

BATCH: 2020

OPEN ENDED LAB (OEL) REPORT

EL-421 EMBEDDED ELECTRONICS

OBJECTIVE:

To build a 5-stage RISC-V pipelined processor capable of executing provided assembly instructions.

PIPELINED RISC-V IMPLEMENTATION:

Our implementation of pipelined RISC-V uses five control registers that are used to divide the data flow in the processor in 5 different stages (i.e. fetch, decode, execute, memory and write back). Furthermore, due to multiple instructions running concurrently, there may be incorrect reading writing or flow data that might induce errors into the results of instructions

So, in order to counter act this, issue we have integrated a Hazard unit that takes certain signals as input and provides hazard control signals according to the status of those signals. These hazard control signals are provided to the control registers in order to perform operations such as “stall” or “flush”.

We tested our implementation of pipelined RISC-V by running the instructions provided in the documentation which are as follows:

Instructions Executed:

```
Main: addi x7, x3, -9
      sw x4, x3(0)
      or x4, x7, x2
      and x5, x3, x4
      beq x5, x7, end
      lw x3, x3(0)
      slt x4, x4, x3
      beq x4, x0, around
around: beq x2, x2, around
end: add x3, x4, x2
```

By observing the output wave forms generated by our design we confirmed that the given instruction set contains multiple hazards.

Hazards Detected:

The provided instruction set contains 5 hazards in total and the distribution of those hazards is given below:

No. Data hazards = **4**

No. of Load word hazards = **1**

No. of Branch Hazards = **0**

Results of instructions:

The previously mentioned register values can be verified by looking at the following signals at their respective clock cycles in the Top-level waveform:

At clock cycle 5: result_W = 3, rd_W=7 *// Instruction 1(addi) is completed*

```
memwrite_M=1, alurest_M=3, writedata_M=1    //Instruction 2(sw) is completed
```

At clock cycle 7: result_W = 7, rd_W=4 // Instruction 3(or) is completed

[illegible]

At clock cycle 8: result_W = 4, rd_W=5 *// Instruction 4(and) is completed*

```
Instruction = 32h'00020263 // Instruction 8(beq2) is completed
```

At clock cycle 10: result_W = 3, rd_W=3 // Instruction 6 (lw) is completed

At clock cycle 12: result_W = 0, rd_W=4 // Instruction 7 (slt) is completed

HAZARD UNIT:

In our design we have found out that the given instruction set has 4 data hazards and 1 load word hazard.

Hazards Detected on the respective clock cycles:

Forward AE=01 at clock cycle 5: rs1_E == rd_W and regwrite_W=1 //data hazard 1 due to x7

Forward BE=10 at clock cycle 6: rs2_E == rd_M and regwrite_M=1 //data hazard 2 due to x4

Forward AE=10 at clock cycle 7: rs1_E == rd_M and regwrite_M=1 //data hazard 3 due to x5

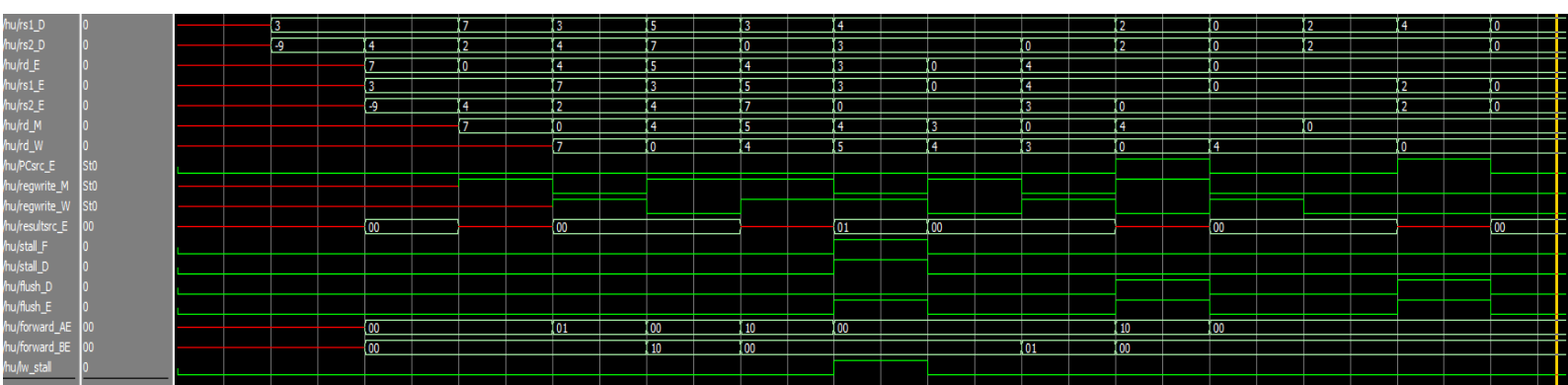
lw_stall = stall_f = stall_D = flush_D = 1 at clock cycle 8: rs2_D == rd_E and resultsrc_E=01 // lw hazard due to x3

Forward BE=01 at clock cycle 10: rs2_E == rd_W and regwrite_W=1 //data hazard 4 due to x4

Complication in 2nd beq instruction(8):

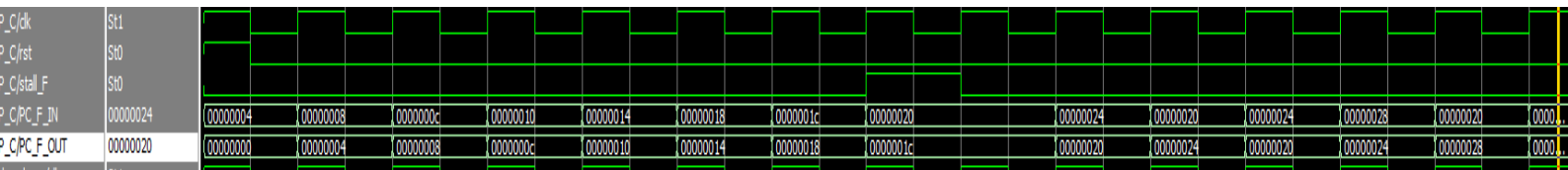
After looking at the waveform of top-level module and hazard unit it can also be seen that the hazard unit signal for branch hazard is also set to high when 2nd beq (8th instruction) is executed, although we are not counting it as a hazard since the instruction jumps to the 3rd beq instantly (9th instruction) which then runs in a loop, hence there was nothing to flush.

Flush_D = Flush_E = 1 at clock cycle 11



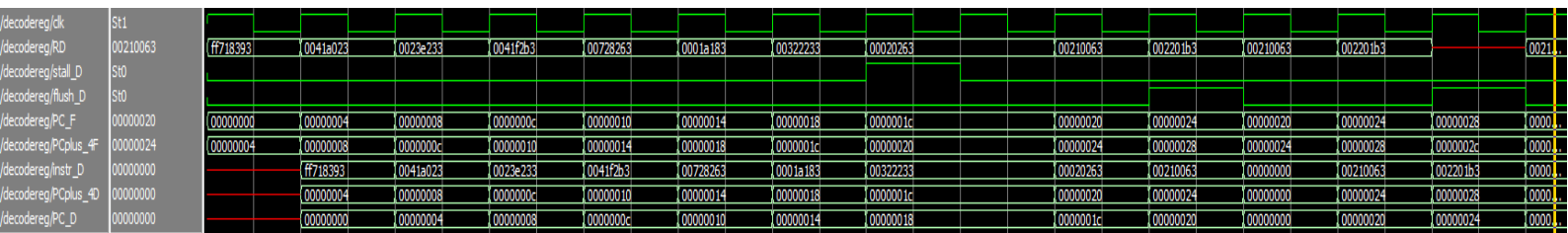
FETCH REGISTER OUTPUT WAVEFORM:

It's a basic program counter which generates the address for instruction memory but has the ability to retain the data whenever the hazard unit signal "stall_F" is pulled high. Overall, it's an enable low register.



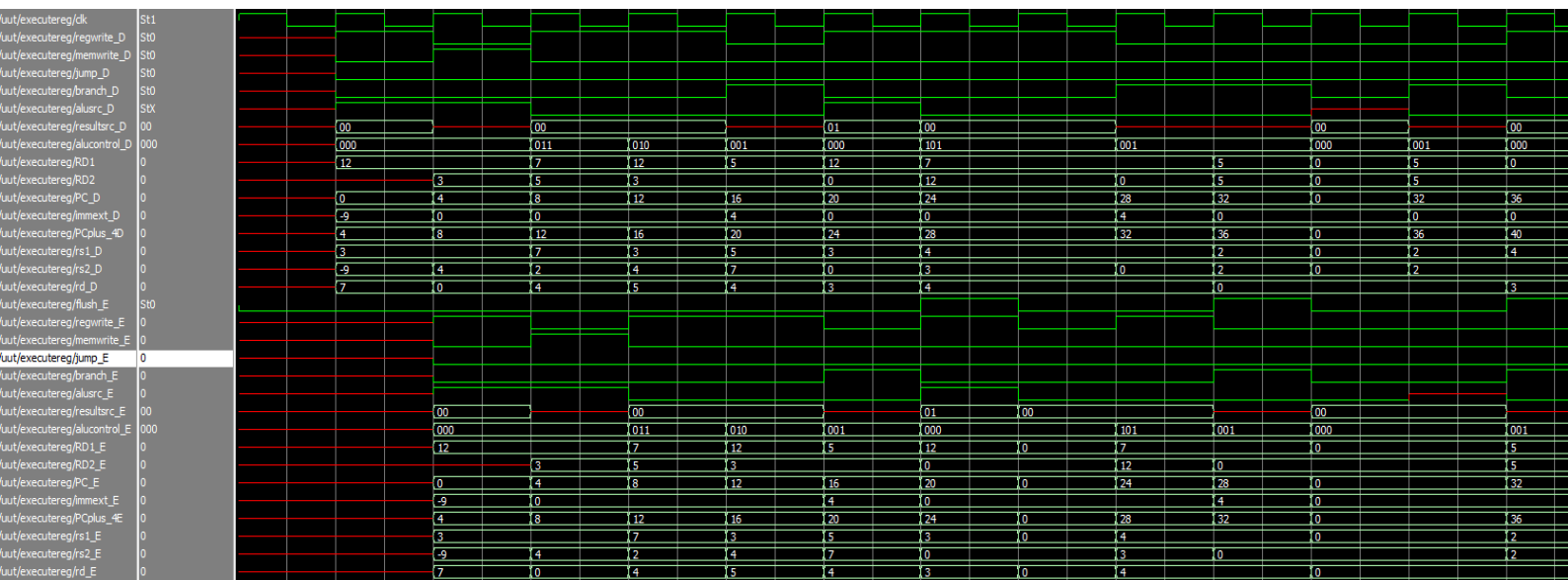
DECODE REGISTER OUTPUT WAVEFORM:

Responsible for passing the fetch stage signals to from fetch to decode stage. This register also has the ability to retain the data whenever hazard unit signal "stall_D" is pulled high. Furthermore, it also takes the hazard unit signal flush_D as an input and whenever it's pulled high it changes all the signal values to 0.



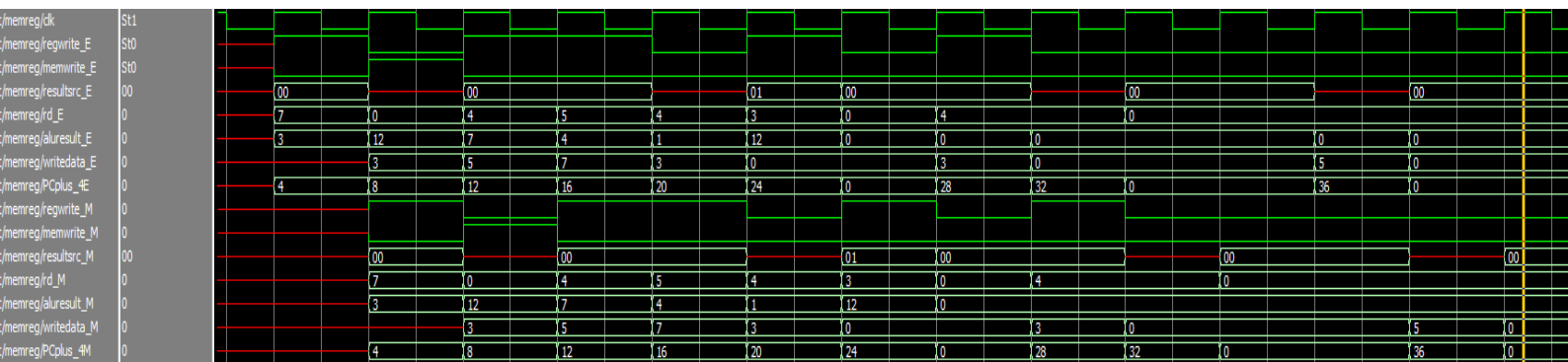
EXECUTION REGISTER OUTPUT WAVEFORM:

Responsible for passing the Decode stage signals from Decode to Execution stage. It takes the hazard unit signal flush_E which assigns the value 0 to the signals, whenever it is pulled high.



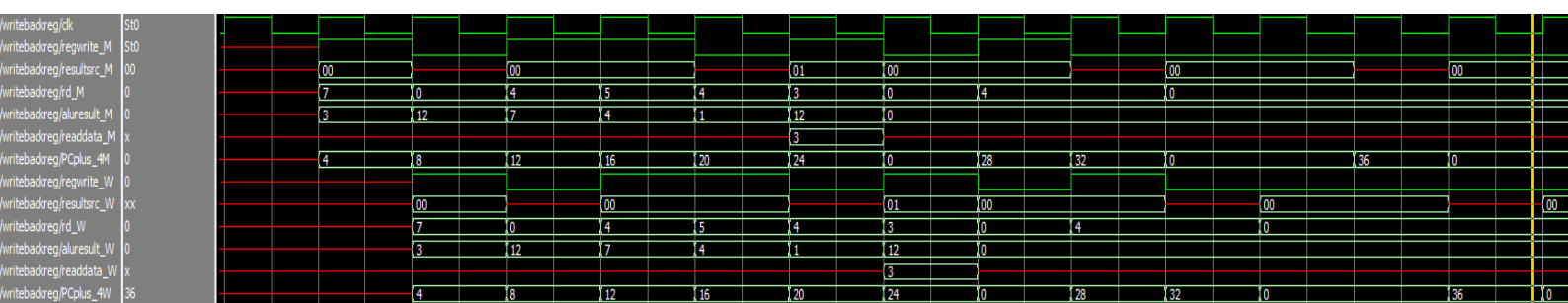
MEMORY REGISTER OUTPUT WAVEFORM:

Responsible for passing the Execution stage signals from execute to memory stage.



WRITE BACK REGISTER OUTPUT WAVEFORM:

Responsible for passing the signals from memory to write back stage



APPENDICES:

```

1 module pipelined_riscv(clk, rst);
2     input clk, rst;
3
4     wire [31:0] PC_F_IN, PC_F_OUT, PCplus_4F, instruction; //FETCH STAGE SIGNALS
5
6     wire [31:0] instr_D, PCplus_4D, PC_D, RD1, RD2, immext_D; //DECODE STAGE SIGNALS
7     wire branch_D, jump_D, memwrite_D, alusrc_D, regwrite_D;
8     wire [1:0] resultsrc_D, immsrc_D, alu_op;
9     wire [2:0] alucontrol_D;
10
11     wire branch_E, jump_E, memwrite_E, alusrc_E, regwrite_E, zero_E; // EXECUTE STAGE SIGNALS
12     wire [1:0] resultsrc_E;
13     wire [2:0] alucontrol_E;
14     wire [4:0] rsl_E, rs2_E, rd_E;
15     wire [31:0] immext_E, PCplus_4E, PC_E, RD1_E, RD2_E, PCTarget_E, aluresult_E;
16     wire [31:0] srcA_E, srcB_E, outputforwadBE;
17     wire PCsrc_E;
18
19     wire regwrite_M, memwrite_M; // MEMORY STAGE SIGNALS
20     wire [1:0] resultsrc_M;
21     wire [4:0] rd_M;
22     wire [31:0] aluresult_M, writedata_M, PCplus_4M, readdata_M;
23
24     wire regwrite_W;
25     wire [1:0] resultsrc_W; //WRITE BACK STAGE SIGNALS
26     wire [31:0] result_W, aluresult_W, readdata_W, PCplus_4W;
27     wire [4:0] rd_W;
28
29     wire stall_F;
30     wire stall_D; // HAZARD CONTROL SIGNALS
31     wire flush_D;
32     wire flush_E;
33     wire [1:0] forward_AE;
34     wire [1:0] forward_BE;
35
36
37
38     //MUX (PCF')
39     mux_pcf pcf_out(
40         .PCplus_4F(PCplus_4F),
41         .PCTarget_E(PCTarget_E),
42         .PCsrc_E(PCsrc_E),
43         .PC_F_IN(PC_F_IN)
44     );
45     //PRAGRAM COUNTER WITH STALL F
46     Program_Counter P_C(
47         .clk(clk),
48         .rst(rst), .stall_F(stall_F),
49         .PC_F_IN(PC_F_IN),
50         .PC_F_OUT(PC_F_OUT)
51     );
52     //ADDER FOR PCPLUS4F
53     pcplus pcplus(
54         .PC_F_OUT(PC_F_OUT),
55         .b(32'd4),
56         .PCplus_4F(PCplus_4F)
57     );
58     //INSTRUCTION MEMORY
59     instruction_memory im(
60         .inst_addr(PC_F_OUT),
61         .instruction(instruction)
62     );
63     //DECODE REGISTER
64     decode_register decodereg(
65         .clk(clk),
66         .RD(instruction),
67         .stall_D(stall_D),
68         .flush_D(flush_D),
69         .PC_F(PC_F_OUT),
70         .PCplus_4F(PCplus_4F),
71         .instr_D(instr_D),
72         .PCplus_4D(PCplus_4D),
73         .PC_D(PC_D)
74     );
75
76     //REGISTER FILE
77     registerFile rf(
78         .WD3(result_W),
79         .rsl(instr_D[19:15]),
80         .rs2(instr_D[24:20]),
81         .rd(rd_W),
82         .write_enable(regwrite_W),
83         .clk(clk),
84         .RD1(RD1),
85         .RD2(RD2)
86     );
87
88     //IMMEDIATE EXTENDER
89     imm_ext immext(
90         .instruction(instr_D[31:7]),
91         .immSrc(immsrc_D),
92         .immext_D(immext_D)
93     );
94
95     //MAIN DECODER(CONTROL UNIT)
96     main_decoder maindec(
97         .opcode(instr_D[6:0]),
98         .branch_D(branch_D),
99         .jump_D(jump_D),
100        .resultsrc_D(resultsrc_D),
101        .memwrite_D(memwrite_D),
102        .alusrc_D(alusrc_D),
103        .immsrc_D(immsrc_D),
104        .regwrite_D(regwrite_D),
105        .alu_op
106    );
107
108    // ALU Decoder
109    alu_decoder alu_dec(
110        .func3(instr_D[14:12]),
111        .func7(instr_D[30]),
112        .aluop_D(alu_op),
113        .alucontrol_D(alucontrol_D)
114    );

```

```

115 // PCSOURCE_E
116 pcsource pcsrc_e(
117     .branch_E(branch_E),
118     .zero_E(zero_E),
119     .jump_E(jump_E),
120     .PCsrc_E(PCsrc_E)
121 );
122 // EXECUTE REGISTER
123 execute_register executereg(
124     .clk(clk),
125     .regwrite_D(regwrite_D), .memwrite_D(memwrite_D), .jump_D(jump_D), .branch_D(branch_D), .alusrc_D(alusrc_D),
126     .resultsrc_D(resultsrc_D),
127     .alucontrol_D(alucontrol_D),
128     .RD1(RD1), .RD2(RD2),
129     .PC_D(PC_D),
130     .rs1_D(instr_D[19:15]),
131     .rs2_D(instr_D[24:20]),
132     .rd_D(instr_D[11:7]),
133     .immext_D(immext_D),
134     .PCplus_4D(PCplus_4D),
135     .flush_E(flush_E),
136     .regwrite_E(regwrite_E), .memwrite_E(memwrite_E), .jump_E(jump_E), .branch_E(branch_E), .alusrc_E(alusrc_E),
137     .resultsrc_E(resultsrc_E),
138     .alucontrol_E(alucontrol_E),
139     .RD1_E(RD1_E), .RD2_E(RD2_E),
140     .PC_E(PC_E),
141     .rs1_E(rs1_E),
142     .rs2_E(rs2_E),
143     .rd_E(rd_E),
144     .immext_E(immext_E),
145     .PCplus_4E(PCplus_4E)
146 );
147 // MULTIPLEXER FORWARD AE
148 forwardAE forwardae(
149     .RD1_E(RD1_E),
150     .result_W(result_W),
151     .alurest_M(alurest_M),
152     .forward_AE(forward_AE),
153     .srcA_E(srcA_E)
154 );
155 // MULTIPLEXER FORWARD BE
156 forwardBE forwardbe(
157     .RD2_E(RD2_E),
158     .result_W(result_W),
159     .alurest_M(alurest_M),
160     .forward_BE(forward_BE),
161     .outputforwadBE(outputforwadBE)
162 );
163 // MUX (ALUSRC E)
164 alusrcE alusrcE(
165     .I1(outputforwadBE),
166     .I2(immext_E),
167     .s(alusrc_E),
168     .srcB_E(srcB_E)
169 );
170 // ADDER FOR PCTARGETE
171 pctargetE pctargetE(
172     .PC_E(PC_E),
173     .immext_E(immext_E),
174     .PCTarget_E(PCTarget_E)
175 );
176 // ALU
177 alu alu(
178     .a(srcA_E),
179     .b(srcB_E),
180     .op(alucontrol_E),
181     .alurest_E(alurest_E),
182     .zero_E(zero_E)
183 );
184 // MEMORY REGISTER
185 memory_register memreg(
186     .clk(clk), .regwrite_E(regwrite_E), .memwrite_E(memwrite_E),
187     .resultsrc_E(resultsrc_E), .rd_E(rd_E),
188     .alurest_E(alurest_E), .writedata_E(outputforwadBE), .PCplus_4E(PCplus_4E),
189     .regwrite_M(regwrite_M), .memwrite_M(memwrite_M),
190     .resultsrc_M(resultsrc_M), .rd_M(rd_M),
191     .alurest_M(alurest_M), .writedata_M(writedata_M), .PCplus_4M(PCplus_4M)
192 );
193 // DATA MEMORY
194 data_memory dm(
195     .clk(clk), .memwrite_M(memwrite_M),
196     .address(alurest_M), .WD(writedata_M),
197     .readdata_M(readdata_M)
198 );
199 // WRITE BACK REGISTER
200 writeback_register writebackreg(
201     .clk(clk), .regwrite_M(regwrite_M),
202     .resultsrc_M(resultsrc_M),
203     .rd_M(rd_M),
204     .alurest_M(alurest_M), .readdata_M(readdata_M), .PCplus_4M(PCplus_4M),
205     .regwrite_W(regwrite_W),
206     .resultsrc_W(resultsrc_W),
207     .rd_W(rd_W),
208     .alurest_W(alurest_W), .readdata_W(readdata_W), .PCplus_4W(PCplus_4W)
209 );
210 // MUX (RESULT SRC W)
211 resultsrcW resultsrcW(
212     .I1(alurest_W), .I2(readdata_W), .I3(PCplus_4W),
213     .s(resultsrc_W),
214     .Out(result_W)
215 );
216 // HAZARD UNIT
217 hazard_unit hu(
218     .rs1_D(instr_D[19:15]), .rs2_D(instr_D[24:20]), .rd_E(rd_E), .rs1_E(rs1_E), .rs2_E(rs2_E), .rd_M(rd_M), .rd_W(rd_W),
219     .PCsrc_E(PCsrc_E), .regwrite_M(regwrite_M), .regwrite_W(regwrite_W),
220     .resultsrc_E(resultsrc_E),
221     .stall_F(stall_F), .stall_D(stall_D),
222     .flush_D(flush_D), .flush_E(flush_E),
223     .forward_AE(forward_AE), .forward_BE(forward_BE)
224 );
225 endmodule
226
227 // MUX FOR PCF
228 module mux_pcf (
229     input [31:0] PCplus_4F, PCTarget_E,

```

```

233         input PCsrc_E,
234         output reg [31:0]PC_F_IN
235     );
236
237     always @(*)
238     begin
239         if (PCsrc_E == 0)
240             PC_F_IN = PCplus_4F;
241         else
242             PC_F_IN = PCtarget_E;
243         end
244     endmodule
245
246 // Program Counter
247 module Program_Counter(
248     input clk, rst, stall_F,
249     input [31:0]PC_F_IN,
250     output reg [31:0]PC_F_OUT
251 );
252     always @(posedge clk)
253     begin
254         if (rst) begin
255             PC_F_OUT <= 0;
256             end
257         else if ( !rst & !stall_F)
258             PC_F_OUT <= PC_F_IN;
259         end
260     endmodule
261
262
263 // ADDER FRO PCPLUS4F
264 module pcplus(
265     input [31:0]PC_F_OUT, b,
266     output reg [31:0]PCplus_4F
267 );
268     always @(*)
269     begin
270         PCplus_4F = PC_F_OUT + b;
271         end
272     endmodule
273
274
275 // Instruction Memory
276 module instruction_memory(
277     input [31:0] inst_addr,
278     output [31:0] instruction
279 );
280
281     reg [7:0] imem [39:0];
282     initial
283     begin
284
285         //Main: addi x7, x3,-9                x7=12-9=3
286         imem[0]= 8'b10010011; //93
287         imem[1]= 8'b10000011; //83
288         imem[2]= 8'b01110001; //71
289         imem[3]= 8'b11111111; //ff
290         //sw x4, x3(0)                Mem[x3+0]=x4    Mem=12    x4=3
291         imem[4]= 8'b00100011; //23
292         imem[5]= 8'b10100000; //a0
293         imem[6]= 8'b01000001; //41
294         imem[7]= 8'b00000000; //00
295         //or x4,x7,x2                x4=x7 | x2        x7=3    x2=5    x4= 7
296         imem[8]= 8'b00110011; //33
297         imem[9]= 8'b11100010; //e2
298         imem[10]= 8'b00100011; //23
299         imem[11]= 8'b00000000; //00
300         //and x5, x3, x4                x5= x3 & x4    x4=7    x3=12    x5= 4
301         imem[12]= 8'b10110011; //b3
302         imem[13]= 8'b11110010; //f2
303         imem[14]= 8'b01000001; //41
304         imem[15]= 8'b00000000; //00
305         //beq x5, x7, end                x5!=x7    x5=4 and x7=3
306         imem[16]= 8'b01100011; //63
307         imem[17]= 8'b10000010; //82
308         imem[18]= 8'b01110010; //72
309         imem[19]= 8'b00000000; //00
310         //lw x3, x3(0)                x3=Mem[x3+0]    mem=12    x3=3
311         imem[20]= 8'b10000011; //83
312         imem[21]= 8'b10100001; //a1
313         imem[22]= 8'b00000001; //01
314         imem[23]= 8'b00000000; //00
315         //slt x4, x4, x3                x4=0    x4 > x3    x4=7    x3=3
316         imem[24]= 8'b00110011; //33
317         imem[25]= 8'b00100010; //22
318         imem[26]= 8'b00110010; //32
319         imem[27]= 8'b00000000; //00
320         //beq x4, x0, around                x4=x0    branch to around
321         imem[28]= 8'b01100011; //63
322         imem[29]= 8'b00000010; //02
323         imem[30]= 8'b00000010; //02
324         imem[31]= 8'b00000000; //00
325         //around: beq x2, x2, around                x2=x2    branch to around    x2=5
326         imem[32]= 8'b01100011; //63
327         imem[33]= 8'b00000000; //00
328         imem[34]= 8'b00100001; //21
329         imem[35]= 8'b00000000; //00
330         //end: add x3,x4,x2                isn't executed
331         imem[36]= 8'b10110011; //b3
332         imem[37]= 8'b00000001; //01
333         imem[38]= 8'b00100010; //22
334         imem[39]= 8'b00000000; //00
335     end
336     assign instruction= {imem[inst_addr+3],imem[inst_addr+2],
337                         imem[inst_addr+1],imem[inst_addr]};
338 endmodule
339
340 // DECODE REGISTER
341 module decode_register (
342     input clk,
343     input [31:0]RD,
344     input stall_D,
345     input flush_D,
346     input [31:0] PC_F,
347     input [31:0] PCplus_4F,
348     output reg [31:0] instr_D,
349     output reg [31:0] PCplus_4D,
350     output reg [31:0] PC_D

```

```

351     );
352
353
354
355     always@(posedge clk) begin
356         if (!stall_D && flush_D==0) begin
357             instr_D <= RD;
358             PC_D <= PC_F;
359             PCplus_4D <= PCplus_4F;
360
361             end
362         else if (flush_D) begin
363             instr_D <= 0 ;
364             PC_D <= 0 ;
365             PCplus_4D <= 0 ;
366
367             end
368         end
369     endmodule
370
371 // Register File
372 module registerFile(
373     input [31:0] WD3,
374     input [4:0] rs1, rs2, rd,
375     input write_enable, clk,
376     output [31:0] RD1, RD2
377 );
378
379     reg [31:0] Registers [31:0];
380     integer i;
381     assign RD1 = Registers[rs1];
382     assign RD2 = Registers[rs2];
383
384     always @(negedge clk)
385     begin
386         if (write_enable)
387             Registers [rd] = WD3;
388     end
389
390     initial
391     begin
392         // for(i=0; i<32; i = i+1)
393         //     Registers[i] = i;
394         Registers[0]=32'd0;
395         Registers[2]=32'd5;
396         Registers[3]=32'd12;
397         Registers[4]=32'd3;
398         Registers[5]=32'd5;
399         Registers[7]=32'd7;
400     end
401 endmodule
402
403 //Immediate Extender
404 module imm_ext(
405     input [31:7] instruction,
406     input [1:0] immSrc,
407     output reg [31:0] immext_D
408 );
409
410     always @(*)
411     begin
412         case (immSrc)
413             2'b00: immext_D = {{20(instruction[31]), instruction[31:20]};
414             2'b01: immext_D = {{20(instruction[31]), instruction[31:25], instruction[11:7]};
415             2'b10: immext_D = {{20(instruction[31]), instruction[7], instruction[30:25], instruction[11:8], 1'b0};
416             default: immext_D = 32'b0;
417         endcase
418     end
419 endmodule
420
421 // Main Decoder
422 module main_decoder (
423     input [6:0] opcode,
424     output reg branch_D,
425     output reg jump_D,
426     output reg [1:0] resultsrc_D,
427     output reg memwrite_D,
428     output reg alusrc_D,
429     output reg [1:0] immsrc_D,
430     output reg regwrite_D,
431     output reg [1:0] alu_op
432 );
433
434 //reg clk;
435 always@(*) begin
436     case (opcode)
437         7'b0000011: begin //lw
438             regwrite_D <= 1;
439             immsrc_D <= 0;
440             alusrc_D <= 1;
441             memwrite_D <= 0;
442             resultsrc_D <= 2'b01;
443             branch_D <= 0;
444             alu_op <= 2'b00;
445             jump_D <= 0;
446         end
447         7'b0100011: begin //sw
448             regwrite_D <= 0;
449             immsrc_D <= 2'b01;
450             alusrc_D <= 1;
451             memwrite_D <= 1;
452             resultsrc_D <= 2'bxx;
453             branch_D <= 0;
454             alu_op <= 2'b0;
455             jump_D <= 0;
456         end
457         7'b0110011: begin //r_type
458             regwrite_D <= 1;
459             immsrc_D <= 2'bxx;
460             alusrc_D <= 0;
461             memwrite_D <= 0;
462             resultsrc_D <= 2'b00;
463             branch_D <= 0;
464             alu_op <= 2'b10;
465             jump_D <= 0;
466         end
467     end
468

```

```

469         7'b1100011: begin //beq
470             regwrite_D <= 0;
471             immsrc_D <= 2'b10;
472             alusrc_D <= 0;
473             memwrite_D <= 0;
474             resultsrc_D <= 2'bxx;
475             branch_D <= 1;
476             alu_op <= 2'b01;
477             jump_D <= 0;
478         end
479
480         7'b1101111: begin //jal
481             regwrite_D <= 1;
482             immsrc_D <= 2'b11;
483             alusrc_D <= 1'bx;
484             memwrite_D <= 0;
485             resultsrc_D <= 2'b10;
486             branch_D <= 0;
487             alu_op <= 2'b11;
488             jump_D <= 1'b1;
489         end
490         7'b0010011: begin //i type
491             regwrite_D <= 1;
492             immsrc_D <= 2'b00;
493             alusrc_D <= 1'b1;
494             memwrite_D <= 0;
495             resultsrc_D <= 2'b00;
496             branch_D <= 0;
497             alu_op <= 2'b00;
498             jump_D <= 1'b0;
499         end
500
501         default: begin
502             regwrite_D <= 1'b0;
503             immsrc_D <= 2'b00;
504             alusrc_D <= 1'bx;
505             memwrite_D <= 1'b0;
506             resultsrc_D <= 2'b00;
507             branch_D <= 1'b0;
508             alu_op <= 2'b00;
509             jump_D <= 1'b0;
510         end
511     endcase
512 end
513 endmodule
514
515
516 // ALU Decoder
517 module alu_decoder(
518     input [2:0] func3,
519     input func7,
520     input [1:0] aluop_D,
521     output reg [2:0] alucontrol_D
522 );
523
524     reg [5:0] conc;
525
526     always@(*) begin
527         conc = {aluop_D, func3, func7};
528     end
529
530     always @(*) begin
531         case (conc)
532             6'b000100: begin
533                 alucontrol_D <= 3'b000; //add
534             end
535             6'b000101: begin
536                 alucontrol_D <= 3'b000; //add
537             end
538             6'b010100: begin
539                 alucontrol_D <= 3'b001; //sub
540             end
541             6'b010000: begin
542                 alucontrol_D <= 3'b001; //beq
543             end
544             6'b010101: begin
545                 alucontrol_D <= 3'b001; //sub
546             end
547             6'b100000: begin
548                 alucontrol_D <= 3'b000; //add
549             end
550             6'b100001: begin
551                 alucontrol_D <= 3'b001; //sub
552             end
553             6'b100100: begin
554                 alucontrol_D <= 3'b101; //set less than
555             end
556             6'b100101: begin
557                 alucontrol_D <= 3'b101; //set less than
558             end
559             6'b101100: begin
560                 alucontrol_D <= 3'b011; //or
561             end
562             6'b101101: begin
563                 alucontrol_D <= 3'b011; //or
564             end
565             6'b101110: begin
566                 alucontrol_D <= 3'b010; //and
567             end
568             6'b101111: begin
569                 alucontrol_D <= 3'b010; //and
570             end
571             default: begin alucontrol_D <= 3'b000; end
572         endcase
573     end
574 endmodule
575
576 // PC Source
577 module pcsource (
578     input branch_E,
579     input zero_E,
580     input jump_E,
581     output reg PCsrc_E=0
582 );
583     reg and_out;
584     always@(*) begin
585         and_out = branch_E & zero_E;
586         PCsrc_E = and_out | jump_E ; end

```

```

587 endmodule
588
589 // EXECUTE REGISTER
590 module execute_register (
591     input clk, regwrite_D, memwrite_D, jump_D, branch_D, alusrc_D,
592     input [1:0] resultsrc_D,
593     input [2:0] alucontrol_D, //decode stage signals
594     input [31:0] RD1, RD2, PC_D, immext_D, PCplus_4D,
595     input [4:0] rs1_D, rs2_D, rd_D,
596     input flush_E,
597     output reg regwrite_E, memwrite_E, jump_E, branch_E, alusrc_E,
598     output reg [1:0] resultsrc_E, //execute stage signals
599     output reg [2:0] alucontrol_E,
600     output reg [31:0] RD1_E, RD2_E, PC_E, immext_E, PCplus_4E,
601     output reg [4:0] rs1_E, rs2_E, rd_E
602 );
603
604 always@(posedge clk) begin
605     if (flush_E) begin //if flush e=1 all execute values = 0
606         regwrite_E <= 0; memwrite_E <= 0;
607         jump_E <= 0; branch_E <= 0;
608         alusrc_E <= 0; resultsrc_E <= 0;
609         alucontrol_E <= 0; RD1_E <= 0;
610         RD2_E <= 0; PC_E <= 0;
611         rs1_E <= 0; rs2_E <= 0;
612         rd_E <= 0; immext_E <= 0;
613         PCplus_4E <= 0;
614     end
615     else begin
616         regwrite_E <= regwrite_D ; memwrite_E <= memwrite_D;
617         jump_E <= jump_D ; branch_E <= branch_D;
618         alusrc_E <= alusrc_D ; resultsrc_E <= resultsrc_D;
619         alucontrol_E <= alucontrol_D ; RD1_E <= RD1;
620         RD2_E <= RD2 ; PC_E <= PC_D;
621         rs1_E <= rs1_D ; rs2_E <= rs2_D;
622         rd_E <= rd_D ; immext_E <= immext_D;
623         PCplus_4E <= PCplus_4D;
624     end
625 end
626
627 end
628
629 endmodule
630
631 // Multiplexer forwardAE
632 module forwardAE(
633     input [31:0]RD1_E, result_W, aluresult_M,
634     input [1:0]forward_AE,
635     output reg [31:0]srcA_E
636 );
637 always @(*)
638 begin
639     if (forward_AE == 00)
640         srcA_E = RD1_E;
641     else if (forward_AE == 01)
642         srcA_E = result_W;
643     else if (forward_AE == 2'b10)
644         srcA_E = aluresult_M;
645     else
646         srcA_E = 0;
647 end
648 endmodule
649
650 // Multiplexer forwardBE
651 module forwardBE(
652     input [31:0]RD2_E, result_W, aluresult_M,
653     input [1:0]forward_BE,
654     output reg [31:0]outputforwadBE
655 );
656 always @(*)
657 begin
658     if (forward_BE == 00)
659         outputforwadBE = RD2_E;
660     else if (forward_BE == 01)
661         outputforwadBE = result_W;
662     else if (forward_BE == 2'b10)
663         outputforwadBE = aluresult_M;
664     else
665         outputforwadBE = 0;
666 end
667 endmodule
668
669 // MUX (ALUSRC E)
670 module alusrcE(
671     input [31:0]I1, I2,
672     input s,
673     output reg [31:0]srcB_E
674 );
675 always @(*)
676 begin
677     if (s == 0)
678         srcB_E = I1;
679     else
680         srcB_E = I2;
681 end
682 endmodule
683
684 //ADDER FOR PCTARGET
685 module pctarget(
686     input [31:0]PC_E, immext_E,
687     output reg [31:0]PCTarget_E
688 );
689 always @(*)
690 begin
691     PCTarget_E = PC_E + immext_E;
692 end
693 endmodule
694
695 // ALU
696 module ALU(
697     input [31:0]a, b,
698     input [2:0]op,
699     output reg [31:0]aluresult_E,
700     output reg zero_E
701 );
702 always @(*)
703 begin
704

```

```

705 if (op == 3'b000)
706     aluresult_E = a + b;
707 else if (op == 3'b001)
708     aluresult_E = a - b;
709 else if (op == 3'b101)
710     aluresult_E = a < b;
711 else if (op == 3'b011)
712     aluresult_E = a | b;
713 else if (op == 3'b010)
714     aluresult_E = a & b;
715 end
716 always @(*)
717 begin
718     if (aluresult_E == 32'd0)
719         zero_E = 1;
720     else
721         zero_E = 0;
722 end
723 endmodule
724
725 //MEMORY REGISTER
726 module memory_register(
727     input clk, regwrite_E, memwrite_E,           //execute stage signals
728     input [1:0] resultsrc_E,
729     input [4:0] rd_E,
730     input [31:0] aluresult_E, writedata_E, PCplus_4E,
731     output reg regwrite_M, memwrite_M,           //memory stage signals
732     output reg [1:0] resultsrc_M,
733     output reg [4:0] rd_M,
734     output reg [31:0] aluresult_M, writedata_M, PCplus_4M
735 );
736     always@(posedge clk) begin
737         regwrite_M <= regwrite_E;    memwrite_M <= memwrite_E;
738         resultsrc_M <= resultsrc_E;    aluresult_M <= aluresult_E;
739         writedata_M <= writedata_E;    rd_M <= rd_E;    PCplus_4M <= PCplus_4E;
740     end
741 endmodule
742
743 //DATA MEMORY
744 module data_memory(
745     input clk, memwrite_M,
746     input [31:0] address_WD,
747     output [31:0] readdata_M
748 );
749     reg [7:0] Memory [31:0];
750
751     assign readdata_M = {Memory[address + 3], Memory[address + 2], Memory[address + 1], Memory[address]};
752
753     always @(posedge clk)
754     begin
755         if (memwrite_M)
756             {Memory[address + 3], Memory[address + 2], Memory[address + 1], Memory[address]} <= WD ;
757     end
758 endmodule
759
760 //WRITE BACK REGISTER
761 module writeback_register(
762     input clk, regwrite_M,
763     input [1:0] resultsrc_M,
764     input [4:0] rd_M,
765     input [31:0] aluresult_M, readdata_M, PCplus_4M,
766     output reg regwrite_W,
767     output reg [1:0] resultsrc_W,
768     output reg [4:0] rd_W,
769     output reg [31:0] aluresult_W, readdata_W, PCplus_4W
770 );
771     always@(posedge clk) begin
772         regwrite_W <= regwrite_M;
773         resultsrc_W <= resultsrc_M;    aluresult_W <= aluresult_M;
774         readdata_W <= readdata_M;    rd_W <= rd_M;    PCplus_4W <= PCplus_4M;
775     end
776 endmodule
777
778 //MUX (RESULT SRC W)
779 module resultsrcw(
780     input [31:0] I1, I2, I3,
781     input [1:0] s,
782     output reg [31:0] Out
783 );
784     always @(*)
785     begin
786         if (s == 00)
787             Out = I1;
788         else if (s == 01)
789             Out = I2;
790         else if (s == 10)
791             Out = I3;
792     end
793 endmodule
794
795 module hazard_unit(
796     input [4:0] rs1_D, rs2_D, rd_E, rs1_E, rs2_E, rd_M, rd_W,
797     input PCsrc_E, regwrite_M, regwrite_W,
798     input [1:0] resultsrc_E,
799     output reg stall_F=0,
800     output reg stall_D=0,
801     output reg flush_D,
802     output reg flush_E,
803     output reg [1:0] forward_AE,
804     output reg [1:0] forward_BE
805 );
806
807     reg lw_stall=0;
808
809     //DATA HAZARD (FROWARDING)
810     always@(*) begin
811         //FORWARD AE
812         if((((rs1_E == rd_M) & regwrite_M) & (rs1_E !=0)) begin //forward from memory stage
813             forward_AE=10;
814         end
815
816         else if((((rs1_E == rd_W) & regwrite_W) & (rs1_E !=0)) begin // forward from writeback stage
817             forward_AE=01;
818         end
819
820         else begin
821             forward_AE=00;
822         end
823     end
824
825     //FORWARD BE
826     if((((rs2_E == rd_M) & regwrite_M) & (rs2_E !=0)) begin //forward from memory stage
827         forward_BE=10;
828     end
829
830     else if((((rs2_E == rd_W) & regwrite_W) & (rs2_E !=0)) begin // forward from writeback stage
831         forward_BE=01;
832     end
833
834     else begin
835         forward_BE=00;
836     end
837
838     // LW hazard
839     always@(*) begin
840         if(resultsrc_E[0] & ((rs1_D == rd_E) | (rs2_D == rd_E))) begin //calculating stall
841             lw_stall<=1;
842         end
843         else begin
844             lw_stall<=0; end
845         flush_F<= lw_stall;
846         stall_D<= lw_stall;
847         stall_F<= lw_stall;
848     end
849
850     // BRANCH CONTROL HAZARD
851     always@(*) begin
852         flush_D <= PCsrc_E;
853         flush_E <= lw_stall | PCsrc_E;
854     end
855 endmodule
856
857 // TEST BENCH (TOP MODULE)
858 module tb_pipelined_riscv();
859     reg clk, rst;
860
861     pipelined_riscv uut(clk, rst);
862
863     initial begin
864         clk = 1;
865     end
866
867     always begin
868         #10 clk = ~clk;
869     end
870
871     initial
872     begin
873         rst = 1;
874         #10 rst = 0;
875     end
876 endmodule

```